Problem 1: Graph Colouring (Sudoku)1. Modeling Sudoku as a GraphTo represent Sudoku as a Constraint Satisfaction Problem (CSP) using graph colouring, we define the graph $G = (V, E)$ as follows:Vertices (V): We define a set of 81 vertices $\{v_{1,1}, v_{1,2}, \ldots, v_{9,9}\}$, where each vertex $v_{i,j}$ corresponds to a cell in the 9×9 grid at row i and column j.Edges (E): An edge exists between two vertices $v_{i,j}$ and $v_{x,y}$ if they belong to the same clique (a subgraph where every node is connected). In Sudoku, these cliques are:Row Clique: $v_{i,j}$ and $v_{i,y}$ for all $j \neq y$.Column Clique: $v_{i,j}$ and $v_{x,j}$ for all $i \neq x$.Block Clique: $v_{i,j}$ and $v_{x,y}$ if they share the same 3×3 sub-grid.Colours (k): We attempt a 9-colouring of the graph. Each colour $\{1, 2, \ldots, 9\}$ represents the digit assigned to that cell.2. Boundary ConditionsThe "Boundary Conditions" in this context refer to the initial state and rules that must be respected:Fixed Vertex Coloring (Pre-assignments): The starting digits provided in the puzzle are "fixed" or "pre-colored" nodes. Their colors cannot be changed during the coloring process.Adjacency Constraint: For any edge $(u, v) \in E$, the color of u must be different from the color of v.Completeness: All 81 vertices must be colored using exactly one of the 9 available colors for the coloring to be considered a solution.

In [1]:
```
import networkx as nx

def create_sudoku_graph():
    G = nx.Graph()

    # 1. Create 81 nodes (0 to 80)
    nodes = [(r, c) for r in range(9) for c in range(9)]
    G.add_nodes_from(nodes)

    # 2. Add edges based on Sudoku rules
    for (r, c) in nodes:
        # Row and Column constraints
        for i in range(9):
            if i != c: G.add_edge((r, c), (r, i)) # Row
            if i != r: G.add_edge((r, c), (i, c)) # Column

        # 3x3 Block constraints
        start_r, start_c = 3 * (r // 3), 3 * (c // 3)
        for br in range(start_r, start_r + 3):
            for bc in range(start_c, start_c + 3):
                if (br, bc) != (r, c):
                    G.add_edge((r, c), (br, bc))
```

```python
    return G

# Analyze the graph
sudoku_graph = create_sudoku_graph()
total_nodes = sudoku_graph.number_of_nodes()
total_edges = sudoku_graph.number_of_edges()

print(f"Sudoku Graph Analysis:")
print(f"Total Vertices: {total_nodes} (9x9 grid)")
print(f"Total Edges (Constraints): {total_edges}")
print(f"Degrees per vertex: {len(list(sudoku_graph.neighbors((0,0))))} (Row: 8, Col: 8, Block: 4 unique extra)")
```

```
Sudoku Graph Analysis:
Total Vertices: 81 (9x9 grid)
Total Edges (Constraints): 810
Degrees per vertex: 20 (Row: 8, Col: 8, Block: 4 unique extra)
```

Problem 2: Bellman-Ford Algorithm1. Algorithm Execution TraceThe Bellman-Ford algorithm finds the shortest path by relaxing all edges $|V| - 1$ times. For this graph, we have 6 vertices $\{S, A, B, C, D, E\}$, so we perform 5 iterations.Initial State: $d(S) = 0$, and all other vertices $d(v) = \infty$.Edge List for Relaxation:
$E = \{(S, E, 8), (S, A, 10), (E, D, 1), (D, C, -1), (D, A, -4), (A, C, 2), (A, B, 1), (C, B, 1)\}$

In [6]:
```python
def run_bellman_ford():
    # 1. Define graph structure
    nodes = ['S', 'A', 'B', 'C', 'D', 'E']
    edges = [
        ('S', 'E', 8), ('S', 'A', 10),
        ('E', 'D', 1),
        ('D', 'C', -1), ('D', 'A', -4),
        ('A', 'C', 2), ('A', 'B', 1),
        ('C', 'B', 1)
    ]

    # 2. Initialization
    distances = {node: float('inf') for node in nodes}
    distances['S'] = 0

    print(f"Initial Distances: {distances}\n")

    # 3. Relaxation process (|V|-1 iterations)
```

```
    for i in range(1, len(nodes)):
        for u, v, weight in edges:
            if distances[u] != float('inf') and distances[u] + weight < distances[v]:
                distances[v] = distances[u] + weight
        print(f"Iteration {i}: {distances}")

    return distances, edges

# Execute the algorithm and store results for further analysis
final_distances, original_edges = run_bellman_ford()
```

```
Initial Distances: {'S': 0, 'A': inf, 'B': inf, 'C': inf, 'D': inf, 'E': inf}

Iteration 1: {'S': 0, 'A': 5, 'B': 6, 'C': 7, 'D': 9, 'E': 8}
Iteration 2: {'S': 0, 'A': 5, 'B': 6, 'C': 7, 'D': 9, 'E': 8}
Iteration 3: {'S': 0, 'A': 5, 'B': 6, 'C': 7, 'D': 9, 'E': 8}
Iteration 4: {'S': 0, 'A': 5, 'B': 6, 'C': 7, 'D': 9, 'E': 8}
Iteration 5: {'S': 0, 'A': 5, 'B': 6, 'C': 7, 'D': 9, 'E': 8}
```

Final Shortest-Path DistancesFrom the code output, the final distances from S are:S: 0E: 8D: 9 ($S \to E \to D$)A: 5 ($S \to E \to D \to A$)C: 7 ($S \to E \to D \to A \to C$)B: 6 ($S \to E \to D \to A \to B$)

Why Dijkstra's Algorithm FailsDijkstra's algorithm is greedy and assumes that once a node is processed, its distance is optimal.Negative Weights: The edge $D \to A = -4$ violates this. Dijkstra would pick $S \to A$ (cost 10) early and mark A as "done."It would never discover that going through E and D first (cost 9) then taking the -4 edge results in a better distance (5).

Negative Cycle DetectionA negative cycle exists if we can relax an edge for the $|V|$-th time

In [9]:
```
def check_for_negative_cycle(dist_results, edges):
    for u, v, weight in edges:
        # If we can still find a shorter path, there is a negative cycle
        if dist_results[u] + weight < dist_results[v]:
            return True
    return False

has_negative_cycle = check_for_negative_cycle(final_distances, original_edges)
print("-" * 30)
print(f"Does the graph contain a negative cycle? {has_negative_cycle}")
```

```
------------------------------
Does the graph contain a negative cycle? False
```

In [ ]: